

Week 8 easier

Week 8: Random Signals. Autocorrelation, cross-correlation, and the response of LTI systems to wide-sense stationary random inputs.

Week 8: Random Signals

Objective: Transition from deterministic signal processing to stochastic frameworks, establishing the mathematical foundation for noise modeling and statistical feature extraction.

Lecture 15: Characterizing Random Signals (75 min)

- **Introduction to Stochastic Processes (15 min):** Defining random variables in discrete time. The concept of an ensemble versus a single realization.
- **Stationarity (20 min):** Strict-sense stationarity (SSS) vs. Wide-Sense Stationarity (WSS). Emphasize why WSS is the standard assumption for most DSP and ML time-series applications.
- **Time-Domain Statistics (25 min):**
 - Expected value and mean: $\mu_x = \mathbb{E}[x[n]]$.
 - Autocorrelation sequence: $r_{xx}[m] = \mathbb{E}[x[n]x[n+m]]$.
 - Cross-correlation: $r_{xy}[m] = \mathbb{E}[x[n]y[n+m]]$.
 - Properties of the autocorrelation matrix (symmetry, positive semi-definiteness).
- **ML Bridge (15 min):** Using cross-correlation for template matching and feature alignment before feeding data into predictive models (connecting to Scikit-Learn preprocessing).

Lecture 16: LTI Systems with Random Inputs (75 min)

- **Mean and Variance Through an LTI System (20 min):** Deriving the output mean $\mu_y = \mu_x \sum_n h[n]$ and the impact of the DC gain.
- **Input-Output Correlation Dynamics (30 min):**
 - Cross-correlation of input and output: $r_{xy}[m] = r_{xx}[m] * h[-m]$.
 - Output autocorrelation: $r_{yy}[m] = r_{xx}[m] * h[m] * h[-m]$, or in terms of the deterministic autocorrelation of the impulse response, $r_{yy}[m] = r_{xx}[m] * r_{hh}[m]$.

- **White Noise & Filtering (25 min):** Defining white noise ($r_{xx}[m] = \sigma^2 \delta[m]$). Analyzing the output of an LTI system driven by white noise (coloring the noise). Application: Generating synthetic noise distributions for data augmentation in PyTorch.
-

Prompt: Can you create slides and python code for these two 75 minute classes of a DSP and ML course for a combined senior and first year Electrical and Computer Engineering class? The DSP textbook is Discrete-Time Signal Processing by Oppenheim and Schaffer. The ML textbook is Machine Learning with PyTorch and Scikit-Learn by Raschka. Add some homework problems with solutions and a weekly quiz with solutions as well.

ChatGPT (Medium):

Done. I created the complete Week 8 Random Signals package.

Lecture slides

- [Lecture 15 — Characterizing Random Signals](#)
- [Lecture 16 — LTI Systems with Random Inputs](#)

Both decks are paced for 75 minutes and include conceptual checks, worked examples, DSP/ML connections, figures, instructor notes, and exit questions.

Python material

- [Jupyter notebook — Week 8 Random Signals DSP + ML Demos](#)
- [Plain Python version](#)

The code covers ensemble realizations, stationarity diagnostics, autocorrelation, correlation matrices and positive semidefiniteness, template matching, Scikit-Learn correlation features, LTI mean/variance propagation, white-noise coloring, PSDs, and PyTorch white/colored-noise augmentation. The notebook was executed successfully end-to-end.

Homework and weekly quiz

- [Homework + Weekly Quiz + Complete Solutions](#)

The packet includes 8 homework problems, complete worked solutions, grading guidance, an 8-question weekly quiz with solutions, and differentiation notes for the mixed first-year/senior audience.

Complete package

- [Download the entire Week 8 package](#)

One technical point I made explicit in Lecture 16: with your Lecture 15 definition

$$r_{xy}[m] = E\{x[n]y[n+m]\},$$

the consistent real-signal input-output relationship is

$$r_{xy}[m] = r_{xx}[m] * h[m].$$

The often-seen $h[-m]$ form corresponds to the alternate convention $E\{x[n+m]y[n]\}$. The slides show both conventions so students do not accidentally mix them.

Week 8, Lecture 15

Explain in more detail what's going on here and what good answers and/or examples are. For all mathematical equations, use standard single dollar signs ... for inline math and double dollar signs

...

for block/display math. Do not use (...) or [...]. Output the raw text without escaping the dollar signs.

Chat GPT (Medium):

Expanded Teaching Notes for Week 8 Lecture 15

Slide 1: Characterizing Random Signals

The main transition in this lecture is from deterministic signals to random processes.

For a deterministic signal, the value of $x[n]$ is known once the signal has been specified. For a random signal, the value at a particular index cannot be predicted exactly. We instead describe repeatable statistical properties.

Two central quantities are the mean

$$\mu_X[n] = E\{X[n]\}$$

and the autocorrelation

$$r_{XX}[m] = E\{X[n]X^*[n+m]\}.$$

For a wide-sense stationary process, the mean does not depend on n , and the autocorrelation depends only on lag m .

A useful opening example is microphone noise. Two recordings made under nominally identical conditions will not contain the same sample values. Nevertheless, they may have approximately the same mean, variance, autocorrelation, and power spectral density.

A good answer to “What can be repeatable when the waveform is not?” is:

“The individual samples are unpredictable, but statistical properties such as mean power, variance, correlation time, and spectral shape may remain repeatable.”

A stronger answer adds:

“These statistics describe the mechanism that produces the signals rather than one particular recording.”

Slide 2: Roadmap and Learning Objectives

Students should leave the lecture able to distinguish four related objects:

- A random variable
- A random process
- One realization
- An ensemble

They should also understand that stationarity concerns invariance of statistics under a shift of the time origin.

A good summary of the lecture is:

“Random-process theory separates the unpredictable sample values from the statistical structure that can be modeled, estimated, and used by DSP or machine-learning algorithms.”

The most important conceptual distinction is between stationarity and ergodicity:

- Stationarity says that certain ensemble statistics do not change with time.
- Ergodicity permits those ensemble statistics to be estimated from a sufficiently long single realization.

These terms should not be used interchangeably.

Slide 3: From a Random Variable to a Random Process

At one fixed time index, the signal value is a random variable. The entire signal is an indexed collection of random variables.

A discrete-time random process can be written as

$$\{X[n]: n \in \mathbb{Z}\}.$$

At a fixed index n_0 ,

$$X[n_0]$$

is one random variable with its own probability distribution.

A realization results when the underlying experiment produces one value for every random variable in the indexed collection:

$$x_i[n].$$

A useful example is a noisy sinusoid:

$$X[n] = A\cos(\omega_0 n + \Phi) + W[n],$$

where A , Φ , or $W[n]$ may be random.

One experiment might produce

$$x_1[n] = 1.1\cos(\omega_0 n + 0.4) + w_1[n],$$

while another produces

$$x_2[n] = 0.9\cos(\omega_0 n + 2.1) + w_2[n].$$

Each curve is a realization of the same process.

A good student answer to “What is random?” is:

“The process is random before the experiment occurs. After one experiment, the realization is an ordinary numerical sequence.”

A common misconception is to call every noisy-looking waveform a random process. A waveform stored in a file is technically one realization. The random process is the probabilistic model that could generate that waveform and other possible waveforms.

Slide 4: Random Process as an Indexed Family

The ensemble is the set of all realizations that the model could generate.

Conceptually, imagine repeating the same measurement experiment many times:

$$x_1[n], x_2[n], \dots, x_M[n].$$

For any fixed $n = n_0$, the values

$$x_1[n_0], x_2[n_0], \dots, x_M[n_0]$$

are samples from the random variable $X[n_0]$.

This gives two different directions for averaging:

1. Ensemble averaging fixes n and averages across experiments.
2. Time averaging fixes one realization and averages across n .

A strong explanation is:

“The ensemble runs vertically across repeated experiments, while time runs horizontally within one experiment.”

Examples include:

- Thermal noise measured repeatedly from the same resistor
- Repeated speech recordings of the same word
- Repeated vibration measurements from a healthy motor
- Different communication packets passing through statistically similar channel conditions

In practice, engineers often have only one or a few finite realizations. This is why assumptions such as stationarity and ergodicity become important.

Slide 5: Ensemble Average and Time Average

The ensemble mean is

$$\mu_X[n] = E\{X[n]\}.$$

If M independent realizations are available, it can be estimated by

$$\hat{\mu}_X[n] = \frac{1}{M} \sum_{i=1}^M x_i[n].$$

If only one realization is available, an engineer may compute a time average:

$$\hat{\mu}_N = \frac{1}{N} \sum_{n=0}^{N-1} x[n].$$

These two averages are not automatically equivalent.

Stationarity can make the ensemble mean independent of n , but stationarity alone does not guarantee that the time average converges to the ensemble mean.

A useful counterexample is

$$X[n] = A,$$

where

$$P(A = 1) = P(A = -1) = \frac{1}{2}.$$

The process is constant over time within each realization. Its ensemble mean is

$$E\{X[n]\} = E\{A\} = 0.$$

But one realization is either $x[n] = 1$ for all n or $x[n] = -1$ for all n . Its time average is therefore either 1 or -1 , not zero.

This process is stationary but not mean ergodic.

A good answer to “Why do we care about ergodicity?” is:

“Because a sensor often gives us one long recording rather than many independent universes. Ergodicity justifies using that recording to estimate ensemble statistics.”

Slide 6: Stationarity

Stationarity asks whether shifting the time origin changes the statistical description.

For a deterministic constant signal, a time shift produces the same waveform. For a stationary random process, a time shift may produce different sample values, but the relevant probability distributions or moments remain unchanged.

A useful physical interpretation is:

“If I start collecting data ten seconds later, do the statistical rules governing the samples remain the same?”

Approximate stationarity may be reasonable for:

- Thermal noise over a short interval
- Background noise during a stable operating condition
- Motor vibration at constant speed and load
- Communication noise over one short packet

It may fail for:

- Speech over an entire sentence
- A machine warming up
- Battery voltage during discharge
- A transient fault
- Traffic measurements across morning and evening periods

Real signals are often locally stationary rather than stationary over an entire recording.

Slide 7: Strict-Sense Stationarity

A process is strict-sense stationary when every joint probability distribution remains unchanged under a common time shift.

For any indices $n_1, \dots, n_k$ and any shift ℓ ,

$$(X[n_1], \dots, X[n_k]) \stackrel{d}{=} (X[n_1 + \ell], \dots, X[n_k + \ell]).$$

The symbol $\stackrel{d}{=}$ means equality in distribution.

This definition applies to all orders. It constrains single-sample distributions, pairs, triples, and every higher-dimensional joint distribution.

An independent and identically distributed process is a straightforward example. If

$$X[n] \sim \mathcal{N}(\mu, \sigma^2)$$

independently for every n , shifting all indices does not change any joint distribution.

A process with time-dependent variance is a non-example:

$$X[n] \sim \mathcal{N}(0, 1 + 0.01n).$$

The distribution at $n = 100$ differs from the distribution at $n = 0$.

A good answer to “Does SSS imply WSS?” is:

“Yes, provided the required first and second moments are finite. Invariance of every joint distribution includes invariance of the first- and second-order statistics.”

A strong answer also notes:

“WSS does not generally imply SSS because matching first- and second-order statistics does not determine every higher-order distribution. For Gaussian processes, however, the mean and covariance determine all finite-dimensional distributions, so the distinction becomes much smaller.”

Slide 8: Wide-Sense Stationarity

A process is WSS when it has a finite second moment and satisfies two conditions.

First, its mean is constant:

$$E\{X[n]\} = \mu_X.$$

Second, its autocorrelation depends only on lag:

$$r_{XX}[n_1, n_2] = E\{X[n_1]X^*[n_2]\} = r_{XX}[n_2 - n_1].$$

The absolute values of n_1 and n_2 do not matter. Only their separation matters.

For example, the statistical relationship between samples separated by five indices must be the same whether the pair is

$$X[10], X[15]$$

or

$$X[1000], X[1005].$$

WSS is sufficient for much of classical linear DSP because LTI-system output means, correlations, and power spectra depend on first- and second-order input statistics.

For a stable LTI system with impulse response $h[n]$, a WSS input produces a constant output mean

$$\mu_Y = \mu_X \sum_k h[k].$$

A good answer to “Why does ML care about approximate stationarity?” is:

“A fixed preprocessing pipeline and a fixed model assume that the statistical relationship between inputs and features remains reasonably stable. Strong nonstationarity can produce distribution shift.”

A stronger answer distinguishes stationarity from identical examples:

“Stationarity does not mean every segment looks the same. It means the relevant probability laws or moments do not depend on absolute time.”

Slide 9: Concept Check on Time-Varying Variance

Suppose

$$E\{X[n]\} = \mu$$

but

$$\text{var}(X[n]) = 1 + 0.01n.$$

This process is not WSS.

The quickest justification uses the zero-lag autocorrelation:

$$r_{XX}[0] = E\{|X[n]|^2\} = \text{var}(X[n]) + |\mu|^2.$$

Therefore,

$$r_{xx}[0] = 1 + 0.01n + |\mu|^2,$$

which depends on n .

A good answer is:

“No. Constant mean satisfies only one WSS condition. The changing variance means the zero-lag autocorrelation depends on absolute time.”

An incomplete answer is:

“No, because the process changes.”

That answer identifies the conclusion but does not identify which WSS requirement fails.

A common misconception is:

“Yes, because the mean is constant.”

Students making this claim have remembered only the first WSS condition.

Slide 10: Time-Domain Statistics

The mean describes the baseline or DC component of the process. Variance describes how strongly the process fluctuates around that baseline. Correlation describes how those fluctuations relate across time.

These statistics answer different questions:

- Mean: Around what level does the process fluctuate?
- Variance: How large are the fluctuations?
- Autocorrelation: How long does the process retain statistical memory?
- Cross-correlation: At what relative delay do two signals resemble each other?

Two processes can have the same mean and variance but very different autocorrelations.

For example, white noise and lowpass-filtered noise can both have zero mean and unit variance. White noise changes independently from sample to sample, while filtered noise changes more smoothly. The autocorrelation distinguishes them.

This is an important ML lesson. Per-sample normalization can make two datasets have similar means and variances without making their temporal structures equivalent.

Slide 11: Mean and Variance

For a WSS process,

$$\mu_X = E\{X[n]\}$$

and

$$\sigma_X^2 = E\{|X[n] - \mu_X|^2\}.$$

Expanding the variance gives

$$\sigma_X^2 = E\{|X[n]|^2\} - |\mu_X|^2.$$

Because

$$r_{XX}[0] = E\{|X[n]|^2\},$$

we obtain

$$r_{XX}[0] = \sigma_X^2 + |\mu_X|^2.$$

For a zero-mean process,

$$r_{XX}[0] = \sigma_X^2.$$

For example, if $\mu_X = 2$ and $\sigma_X^2 = 9$, then

$$r_{XX}[0] = 9 + 2^2 = 13.$$

A good answer to “Is $r_{XX}[0]$ always the variance?” is:

“No. It is the mean-square value. It equals the variance only when the process has zero mean.”

For finite data, the sample mean is

$$\hat{\mu}_X = \frac{1}{N} \sum_{n=0}^{N-1} x[n].$$

A corresponding variance estimate is

$$\hat{\sigma}_X^2 = \frac{1}{N-1} \sum_{n=0}^{N-1} |x[n] - \hat{\mu}_X|^2.$$

These estimates are themselves random because they depend on the particular realization and its finite length.

Slide 12: Autocorrelation

Using the convention in the slides,

$$r_{XX}[m] = E\{X[n]X^*[n+m]\}.$$

The conjugate matters for complex-valued signals.

Important properties include

$$r_{XX}[-m] = r_{XX}^*[m]$$

and

$$|r_{XX}[m]| \leq r_{XX}[0].$$

For a real-valued WSS process,

$$r_{XX}[-m] = r_{XX}[m],$$

so the autocorrelation is even.

For zero-mean white noise with variance σ_W^2 ,

$$r_{WW}[m] = \sigma_W^2 \delta[m].$$

If white noise has nonzero mean μ_W , then

$$r_{WW}[m] = |\mu_W|^2 + \sigma_W^2 \delta[m].$$

A useful filtered-noise example is

$$Y[n] = W[n] + 0.5W[n-1],$$

where $W[n]$ is zero-mean white noise with variance σ_W^2 .

Then

$$r_{YY}[0] = (1^2 + 0.5^2)\sigma_W^2 = 1.25\sigma_W^2,$$

and

$$r_{YY}[1] = r_{YY}[-1] = 0.5\sigma_W^2.$$

For $|m| > 1$,

$$r_{YY}[m] = 0.$$

This shows how filtering creates correlation between neighboring samples.

A good interpretation is:

“White noise has no second-order memory beyond lag zero. Filtering spreads that dependence across several lags.”

Slide 13: Cross-Correlation

Using the slide convention,

$$r_{XY}[m] = E\{X[n]Y^*[n+m]\}.$$

Cross-correlation measures similarity between one process and a shifted version of another.

Suppose

$$Y[n] = X[n-d].$$

Then

$$r_{XY}[m] = E\{X[n]X^*[n+m-d]\} = r_{XX}[m-d].$$

If $r_{XX}[m]$ has its maximum at $m = 0$, then $r_{XY}[m]$ has its maximum at

$$m = d.$$

Thus, under this convention, the peak identifies the delay of Y relative to X .

A strong student answer should always state the correlation convention before interpreting the sign of the peak lag. Different textbooks and software packages can reverse the lag definition.

A useful normalized form is

$$\rho_{XY}[m] = \frac{r_{XY}[m]}{\sqrt{r_{XX}[0]r_{YY}[0]}}.$$

Normalized correlation reduces sensitivity to overall amplitude changes.

Applications include:

- Packet synchronization
- Radar and sonar delay estimation
- Time-delay estimation between microphones
- ECG beat alignment
- Template-based fault detection
- Alignment of audio events before feature extraction

A good answer to “Why normalize the correlation?” is:

“Ordinary correlation grows when either signal amplitude grows. Normalization makes the score reflect waveform similarity more than raw energy.”

Slide 14: Autocorrelation Matrix

For a finite random vector

$$\mathbf{x} = [X[0] \quad X[1] \quad \cdots \quad X[N-1]]^T,$$

the correlation matrix is

$$\mathbf{R}_X = E\{\mathbf{x}\mathbf{x}^H\}.$$

Its entries are

$$R_X[i, j] = E\{X[i]X^*[j]\}.$$

For a WSS process,

$$R_X[i, j] = r_{XX}[j - i].$$

Because entries depend only on $j - i$, the matrix is Toeplitz.

For a real process with

$$r_{XX}[0] = 2, \quad r_{XX}[1] = 0.8, \quad r_{XX}[2] = 0.2,$$

a three-sample correlation matrix is

$$\mathbf{R}_X = \begin{bmatrix} 2 & 0.8 & 0.2 \\ 0.8 & 2 & 0.8 \\ 0.2 & 0.8 & 2 \end{bmatrix}.$$

The matrix must also be symmetric for real processes or Hermitian for complex processes:

$$\mathbf{R}_X = \mathbf{R}_X^H.$$

The matrix is positive semidefinite:

$$\mathbf{a}^H \mathbf{R}_X \mathbf{a} \geq 0$$

for every vector $\mathbf{a}$.

A good answer describing the Toeplitz property is:

“Every diagonal contains the same correlation lag. The main diagonal contains $r_{XX}[0]$, the first upper diagonal contains $r_{XX}[1]$, and so forth.”

A strong answer also recognizes that an arbitrary symmetric Toeplitz matrix is not automatically a valid correlation matrix. It must also be positive semidefinite.

Slide 15: Why the Correlation Matrix Is Positive Semidefinite

Start with

$$\mathbf{R}_X = E\{\mathbf{x}\mathbf{x}^H\}.$$

For any deterministic vector $\mathbf{a}$,

$$\mathbf{a}^H \mathbf{R}_X \mathbf{a} = \mathbf{a}^H E\{\mathbf{x}\mathbf{x}^H\} \mathbf{a}.$$

Move the deterministic vectors inside the expectation:

$$\mathbf{a}^H \mathbf{R}_X \mathbf{a} = E\{\mathbf{a}^H \mathbf{x}\mathbf{x}^H \mathbf{a}\}.$$

Because

$$\mathbf{x}^H \mathbf{a} = (\mathbf{a}^H \mathbf{x})^*,$$

we have

$$\mathbf{a}^H \mathbf{R}_X \mathbf{a} = E\{|\mathbf{a}^H \mathbf{x}|^2\}.$$

A squared magnitude cannot be negative, so

$$\mathbf{a}^H \mathbf{R}_X \mathbf{a} \geq 0.$$

A good answer is:

“The quadratic form equals the expected output energy of a linear combination of the samples, and energy cannot be negative.”

A stronger DSP interpretation is:

“If $\mathbf{a}$ represents FIR coefficients, then $\mathbf{a}^H \mathbf{R}_X \mathbf{a}$ is the mean-square output of that filter applied to the random vector.”

Consequently, every eigenvalue of $\mathbf{R}_X$ must satisfy

$$\lambda_k \geq 0.$$

Small negative eigenvalues sometimes appear in estimated correlation matrices because of finite data, inconsistent estimation, or numerical roundoff.

Slide 16: ML Bridge Through Alignment

Many classifiers should recognize the same event even when it occurs at different positions in the observation window.

Suppose every recording contains the same pulse, but the pulse starts at a different sample. Directly flattening the recordings into feature vectors forces the classifier to learn a separate pattern for every possible shift.

Correlation provides an alignment estimate before classification.

This process introduces a useful invariance:

“The class should remain unchanged when the event shifts in time.”

Examples include:

- A spoken keyword occurring at different locations in an audio buffer
- An ECG heartbeat appearing at different offsets
- A machinery impact occurring at different points in a measurement window
- A known preamble arriving at an unknown communication delay

Alignment is appropriate only when the time shift is a nuisance variable. If the absolute event time carries class information, removing it can hurt performance.

Slide 17: Template Matching as Preprocessing

Let $s[n]$ be a known template and $x[n]$ an observation.

A finite-data correlation score can be written as

$$c[m] = \sum_n x[n]s^*[n-m].$$

The estimated lag is

$$\hat{m} = \operatorname{argmax}_m |c[m]|.$$

The processing pipeline is then:

1. Compute the correlation between the observation and template.
2. Locate the peak lag.
3. Shift or crop the observation relative to that lag.
4. Extract features from the aligned observation.
5. Pass the features to the classifier.

A good answer to “Is correlation the classifier?” is:

“No. Correlation estimates how strongly and where a known template occurs. The resulting score or lag can be used as a feature, or the lag can be used to align the input before classification.”

A stronger answer adds:

“If each class has a distinctive known template, a bank of correlation scores can itself form a simple matched-filter classifier. In this slide, however, correlation serves primarily as preprocessing.”

Potential failure cases include:

- The event is stretched or compressed rather than shifted.

- The template shape changes substantially between examples.
 - Multiple copies of the event occur.
 - A stronger interfering feature produces a larger correlation peak.
 - The event amplitude or baseline varies greatly.
 - The correlation peak is broad or ambiguous.
-

Slide 18: Coding Prompt for Lag Estimation

A typical SciPy implementation is:

```
import numpy as np
from scipy import signal

correlation = signal.correlate(
    observation,
    template,
    mode="full"
)

lags = signal.correlation_lags(
    len(observation),
    len(template),
    mode="full"
)

estimated_lag = lags[np.argmax(correlation)]
```

If the template may appear with reversed sign, use

```
estimated_lag = lags[np.argmax(np.abs(correlation))]
```

but only when sign reversal is physically acceptable.

A stronger implementation subtracts the local mean and normalizes energy so that the score reflects shape rather than amplitude.

Questions and good answers:

What happens as SNR decreases?

The peak becomes less distinct relative to random correlation fluctuations. The lag estimate becomes more variable and can jump to an incorrect local maximum.

What if the observation amplitude doubles?

Ordinary correlation approximately doubles. The peak location may remain unchanged, but the score is no longer directly comparable across examples. Normalized correlation helps.

What if the signal is stretched?

Ordinary correlation assumes a shift-only model. A stretched waveform may produce a smaller and broader peak. Possible solutions include a bank of scaled templates, resampling, dynamic time warping, or a learned alignment model.

Why predict the lag before running the code?

The prediction forces students to understand the lag convention. Otherwise, an apparently correct numerical result can hide a sign mistake.

A useful test is to construct an observation with a known insertion point:

```
rng = np.random.default_rng(15)

template = signal.windows.hann(31)
observation = 0.25 * rng.standard_normal(200)

true_lag = 83
observation[true_lag:true_lag + len(template)] += template
```

Students should predict that the correlation peak will occur near `true_lag`, subject to the exact indexing convention.

Slide 19: Lecture Wrap-Up

The main framework is:

- A random process describes an ensemble of possible signals.
- A sensor records one realization.
- Stationarity describes invariance of statistical laws or moments under time shifts.
- Ergodicity justifies estimating ensemble statistics from one long realization.
- Autocorrelation describes second-order memory.
- Cross-correlation describes lagged similarity.
- Correlation matrices collect multiple lag relationships.
- Correlation-based alignment can remove irrelevant timing variation before ML.

For the exit question,

$$r_{XX}[n_1, n_2] = \exp(-0.2|n_2 - n_1|).$$

This autocorrelation depends only on the lag

$$m = n_2 - n_1.$$

If the mean is also constant, the process is WSS.

A good answer is:

“The mean is constant, and the autocorrelation depends only on $n_2 - n_1$, not on the absolute values of n_1 and n_2 .”

A stronger answer adds:

“The zero-lag value is $r_{XX}[0] = 1$, and the exponential decay indicates that dependence becomes weaker as the sample separation increases.”

The correlation falls to approximately e^{-1} when

$$0.2|m| = 1,$$

so a rough correlation length is

$$|m| \approx 5 \text{ samples.}$$

That gives a physical interpretation of the exponent.

Slide 20: References and Instructor Resources

The Oppenheim and Schaffer material provides the mathematical framework for random signals, correlation, LTI systems, and eventually power spectral density.

The Raschka connection concerns preprocessing and the statistical consistency of ML inputs.

The accompanying notebook should reinforce four ideas:

1. Generate many realizations and distinguish ensemble averages from time averages.
2. Compare white noise with filtered noise through autocorrelation.
3. Construct a Toeplitz correlation matrix and inspect its eigenvalues.
4. Estimate the delay of a noisy template using cross-correlation.

A useful extension for graduate students is to connect autocorrelation to the power spectral density through the Wiener-Khinchin relationship:

$$S_{XX}(e^{j\omega}) = \sum_{m=-\infty}^{\infty} r_{XX}[m]e^{-j\omega m}.$$

This says that the power spectral density is the DTFT of the autocorrelation.

For zero-mean white noise,

$$r_{WW}[m] = \sigma_W^2 \delta[m],$$

so

$$S_{WW}(e^{j\omega}) = \sigma_W^2.$$

Its power is spread uniformly across discrete-time frequency.

For a process with slowly decaying autocorrelation, the power spectrum is typically concentrated in a narrower frequency region. This connection prepares students for power spectral estimation, Welch's method, and the later transition to spectrograms.

Week 8, Lecture 16

Explain in more detail what's going on here and what good answers and/or examples are. For all mathematical equations, use standard single dollar signs ... for inline math and double dollar signs

...

for block/display math. Do not use (...) or [...]. Output the raw text without escaping the dollar signs.

Chat GPT (Medium):

Expanded Teaching Notes for Week 8 Lecture 16

Slide 1: LTI Systems with Random Inputs

The central question is different from ordinary deterministic filtering.

For a deterministic input, we may try to calculate the exact output waveform:

$$y[n] = \sum_k h[k]x[n-k].$$

For a random input, the exact future samples are generally unknown. Instead, we predict output statistics such as:

- Output mean
- Output variance
- Output autocorrelation
- Output power spectral density

The main relationships are

$$\mu_Y = \mu_X \sum_k h[k]$$

and, for a real-valued filter,

$$r_{YY}[m] = r_{XX}[m] * h[m] * h[-m].$$

A good opening explanation is:

“We cannot predict the particular noise waveform that will leave the filter, but we can predict its average value, power, correlation structure, and spectrum.”

A stronger answer adds:

“The LTI system transforms first-order and second-order statistics according to the same convolution structure that transforms deterministic signals.”

Slide 2: Roadmap and Learning Objectives

The lecture has four main ideas.

First, the output mean depends on the filter’s DC gain.

Second, output variance depends on both the filter and the input covariance. Mean information alone does not determine output variance.

Third, an LTI filter transforms the autocorrelation and power spectral density of a random input in a predictable way.

Finally, these ideas guide realistic noise augmentation for machine-learning systems.

A useful unifying question is:

“What properties of the input noise will the filter preserve, remove, or introduce?”

By the end of the lecture, a student should be able to answer:

- Does a filter remove a nonzero input mean?
- Does the filter increase or decrease noise variance?
- Does white noise remain white?
- What output frequencies contain most of the noise power?
- Does a proposed ML augmentation resemble the nuisance process encountered during deployment?

Slide 3: Mean and Variance Through an LTI System

Let the output of an LTI system be

$$Y[n] = \sum_k h[k]X[n-k].$$

Because expectation is linear and $h[k]$ is deterministic,

$$E\{Y[n]\} = E\left\{\sum_k h[k]X[n-k]\right\}.$$

Move the expectation through the sum:

$$E\{Y[n]\} = \sum_k h[k]E\{X[n-k]\}.$$

If $X[n]$ is WSS, then

$$E\{X[n-k]\} = \mu_X$$

for every k . Therefore,

$$\mu_Y = \mu_X \sum_k h[k].$$

The key point is that the mean behaves like a DC signal. Variance requires second-order information because it measures fluctuations around that mean.

A useful interpretation is:

- The mean passes through the filter according to its response at DC.
- The random fluctuations pass through according to the entire frequency response and correlation structure.

Slide 4: Output Mean and DC Gain

The frequency response of an LTI system is

$$H(e^{j\omega}) = \sum_k h[k]e^{-j\omega k}.$$

At DC, $\omega = 0$, so

$$H(e^{j0}) = \sum_k h[k].$$

Consequently,

$$\mu_Y = \mu_X H(e^{j0}).$$

This relationship has a direct physical interpretation.

Case 1: Unit DC gain

If

$$\sum_k h[k] = 1,$$

then

$$\mu_Y = \mu_X.$$

The filter preserves the baseline level.

For example, a moving-average filter

$$h[n] = \frac{1}{3}(\delta[n] + \delta[n-1] + \delta[n-2])$$

has

$$\sum_k h[k] = 1.$$

It smooths fluctuations but preserves a constant input.

Case 2: Zero DC gain

If

$$\sum_k h[k] = 0,$$

then

$$\mu_Y = 0$$

regardless of the constant input mean.

The first-difference filter

$$h[n] = \delta[n] - \delta[n-1]$$

has zero DC gain.

For a constant input $x[n] = C$,

$$y[n] = C - C = 0.$$

Case 3: DC gain greater than one

If

$$\sum_k h[k] = 2,$$

then

$$\mu_Y = 2\mu_X.$$

The filter doubles the mean component.

A good answer to “Why does only the DC gain determine the output mean?” is:

“The mean is constant with time, so it behaves like a zero-frequency component. An LTI system multiplies that component by $H(e^{j0})$.”

A common misconception is that a lowpass filter always preserves the mean. It preserves the mean only if its DC gain equals one.

Slide 5: Concept Check on the First-Difference Filter

The input has mean

$$\mu_X = 3$$

and

$$h[n] = \delta[n] - \delta[n - 1].$$

The sum of the impulse response is

$$\sum_k h[k] = 1 - 1 = 0.$$

Therefore,

$$\mu_Y = 3(1 - 1) = 0.$$

A good answer is:

“The output mean is zero because the first-difference filter has zero DC gain.”

A stronger answer relates this directly to the output:

$$Y[n] = X[n] - X[n - 1].$$

For a WSS input,

$$E\{Y[n]\} = E\{X[n]\} - E\{X[n - 1]\} = \mu_X - \mu_X = 0.$$

An incomplete answer is:

“The output is zero.”

The output is not generally the all-zero random sequence. Only its mean is zero. Random differences remain.

For example, if

$$X[n] = 3 + W[n],$$

then

$$Y[n] = W[n] - W[n - 1].$$

The constant value 3 disappears, but the random component remains.

Slide 6: Output Variance

Define the centered input and output processes as

$$\tilde{X}[n] = X[n] - \mu_X$$

and

$$\tilde{Y}[n] = Y[n] - \mu_Y.$$

Because the system is linear,

$$\tilde{Y}[n] = \sum_k h[k] \tilde{X}[n - k].$$

The output variance is

$$\sigma_Y^2 = E\{|\tilde{Y}[n]|^2\}.$$

Substitute the convolution expression:

$$\sigma_Y^2 = E\left\{\left(\sum_k h[k] \tilde{X}[n - k]\right)\left(\sum_\ell h[\ell] \tilde{X}[n - \ell]\right)^*\right\}.$$

Expanding gives

$$\sigma_Y^2 = \sum_k \sum_\ell h[k] h^*[\ell] E\{\tilde{X}[n - k] \tilde{X}^*[n - \ell]\}.$$

Under the correlation convention used in these lectures,

$$c_{XX}[m] = E\{\tilde{X}[n] \tilde{X}^*[n + m]\}.$$

The difference between the two sample indices is $k - \ell$, so

$$\sigma_Y^2 = \sum_k \sum_\ell h[k] h^*[\ell] c_{XX}[k - \ell].$$

This expression shows why the input variance alone may not be enough. Off-zero covariance values describe how different input samples combine through the filter.

A good answer to “Why does correlation matter?” is:

“The output is a weighted sum of several input samples. Its variance depends on whether those samples tend to reinforce or cancel one another.”

White-noise special case

For zero-mean white noise,

$$c_{XX}[m] = \sigma_X^2 \delta[m].$$

Only terms with $k = \ell$ survive, giving

$$\sigma_Y^2 = \sigma_X^2 \sum_k |h[k]|^2.$$

The output variance equals the input variance multiplied by the energy of the impulse response.

Slide 7: Input-Output Correlation Dynamics

Correlation lets us predict how the output relates statistically to the input and to delayed versions of itself.

For deterministic signals, convolution describes waveform transformation.

For random processes, convolution also describes the transformation of correlation sequences.

The two important quantities are:

$$r_{XY}[m] = E\{X[n]Y^*[n + m]\}$$

and

$$r_{YY}[m] = E\{Y[n]Y^*[n + m]\}.$$

Input-output correlation answers:

“How strongly is the output at a future or delayed index related to the current input?”

Output autocorrelation answers:

“How strongly are two output samples separated by m indices related?”

These relationships become especially useful when calculating output noise power and power spectral density.

Slide 8: Correlation Convention

The lecture uses

$$r_{XY}[m] = E\{X[n]Y^*[n+m]\}.$$

An alternate convention is

$$\tilde{r}_{XY}[m] = E\{X[n+m]Y^*[n]\}.$$

These are closely related, but the required reversal changes.

With the lecture convention and

$$Y[n] = h[n] * X[n],$$

the input-output correlation is

$$r_{XY}[m] = r_{XX}[m] * h^*[m].$$

Under the alternate convention, a reversed filter appears:

$$\tilde{r}_{XY}[m] = r_{XX}[m] * h_{\text{rev}}^*[m],$$

where

$$h_{\text{rev}}[m] = h[-m].$$

A good answer to “Why do different books show different formulas?” is:

“They may place the lag on different arguments in the cross-correlation definition. The formulas agree after the lag convention and required reversal are handled consistently.”

A strong answer verifies the formula with the identity system. If

$$h[n] = \delta[n],$$

then the result should reduce to

$$r_{XY}[m] = r_{XX}[m].$$

Another useful check is a pure delay:

$$h[n] = \delta[n-d].$$

Then

$$Y[n] = X[n - d]$$

and

$$r_{XY}[m] = r_{XX}[m - d].$$

The peak occurs at $m = d$ when the input autocorrelation peaks at zero.

Slide 9: Derivation of the Input-Output Correlation

Start with

$$Y[n + m] = \sum_k h[k] X[n + m - k].$$

Taking the complex conjugate gives

$$Y^*[n + m] = \sum_k h^*[k] X^*[n + m - k].$$

Now substitute into the correlation definition:

$$r_{XY}[m] = E\{X[n]Y^*[n + m]\}.$$

Therefore,

$$r_{XY}[m] = E\left\{X[n] \sum_k h^*[k] X^*[n + m - k]\right\}.$$

Move the expectation through the deterministic sum:

$$r_{XY}[m] = \sum_k h^*[k] E\{X[n] X^*[n + m - k]\}.$$

Recognize the input autocorrelation:

$$E\{X[n] X^*[n + m - k]\} = r_{XX}[m - k].$$

Thus,

$$r_{XY}[m] = \sum_k h^*[k] r_{XX}[m - k].$$

This is convolution:

$$r_{XY}[m] = (r_{XX} * h^*)[m].$$

For a real-valued impulse response,

$$h^*[k] = h[k],$$

so

$$r_{XY}[m] = (r_{XX} * h)[m].$$

A good explanation is:

“The system filters the input autocorrelation with the same impulse response, with conjugation included for a complex filter.”

Slide 10: Output Autocorrelation

The output autocorrelation is

$$r_{YY}[m] = E\{Y[n]Y^*[n+m]\}.$$

Substituting the two convolution sums gives

$$r_{YY}[m] = \sum_k \sum_\ell h[k]h^*[\ell]r_{XX}[m+k-\ell].$$

For a real-valued filter, this can be written as

$$r_{YY}[m] = r_{XX}[m] * h[m] * h[-m].$$

Define the deterministic autocorrelation of the impulse response as

$$r_{hh}[m] = \sum_k h[k]h^*[k+m].$$

For a real-valued filter, this correlation is even, and the output relationship can be written as

$$r_{YY}[m] = r_{XX}[m] * r_{hh}[m].$$

The interpretation is that the filter acts once on each member of the product inside the output autocorrelation. This accounts for the original filter and its reversed counterpart.

At zero lag,

$$r_{YY}[0] = E\{|Y[n]|^2\}.$$

This is the output mean-square value. The output variance is

$$\sigma_Y^2 = r_{YY}[0] - |\mu_Y|^2.$$

For a zero-mean output,

$$\sigma_Y^2 = r_{YY}[0].$$

Complex-filter notation nuance

With the lecture convention

$$r_{XY}[m] = E\{X[n]Y^*[n+m]\},$$

a fully consistent complex-filter form is

$$r_{YY} = r_{XX} * h^* * h_{\text{rev}}.$$

For real-valued filters, this reduces to the form emphasized in the slides:

$$r_{YY} = r_{XX} * h * h_{\text{rev}}.$$

Since most filters used in the examples are real, the distinction does not change those results.

Slide 11: Does White Noise Remain White?

Let

$$h[n] = \delta[n] + 0.5\delta[n-1].$$

Then

$$Y[n] = X[n] + 0.5X[n-1].$$

Suppose $X[n]$ is zero-mean white noise with variance σ_X^2 .

Adjacent output samples are

$$Y[n] = X[n] + 0.5X[n-1]$$

and

$$Y[n+1] = X[n+1] + 0.5X[n].$$

Both contain $X[n]$. Therefore, the adjacent outputs are correlated.

The output variance is

$$r_{YY}[0] = \sigma_X^2(1^2 + 0.5^2) = 1.25\sigma_X^2.$$

The lag-one correlation is

$$r_{YY}[1] = 0.5\sigma_X^2.$$

Similarly,

$$r_{YY}[-1] = 0.5\sigma_X^2.$$

For $|m| > 1$,

$$r_{YY}[m] = 0.$$

A good answer is:

“No. The input samples are uncorrelated, but neighboring output samples share an input sample. The filter therefore introduces correlation.”

A stronger answer gives the output autocorrelation:

$$r_{YY}[m] = \sigma_X^2 [1.25\delta[m] + 0.5\delta[m-1] + 0.5\delta[m+1]].$$

Slide 12: Matrix View of Covariance Propagation

For a finite block of samples, convolution can be represented by a matrix:

$$\mathbf{y} = \mathbf{H}\mathbf{x}.$$

Let the centered input vector have covariance

$$\mathbf{C}_X = E\{(\mathbf{x} - \boldsymbol{\mu}_X)(\mathbf{x} - \boldsymbol{\mu}_X)^H\}.$$

The centered output is

$$\tilde{\mathbf{y}} = \mathbf{H}\tilde{\mathbf{x}}.$$

Therefore,

$$\mathbf{C}_Y = E\{\tilde{\mathbf{y}}\tilde{\mathbf{y}}^H\}.$$

Substitute $\tilde{\mathbf{y}} = \mathbf{H}\tilde{\mathbf{x}}$:

$$\mathbf{C}_Y = E\{\mathbf{H}\tilde{\mathbf{x}}\tilde{\mathbf{x}}^H\mathbf{H}^H\}.$$

Since $\mathbf{H}$ is deterministic,

$$\mathbf{C}_Y = \mathbf{H}\mathbf{C}_X\mathbf{H}^H.$$

For white input noise,

$$\mathbf{C}_X = \sigma_X^2 \mathbf{I}.$$

Then

$$\mathbf{C}_Y = \sigma_X^2 \mathbf{H}\mathbf{H}^H.$$

Unless $\mathbf{H}\mathbf{H}^H$ is proportional to the identity, the output covariance contains nonzero off-diagonal entries. Those entries represent correlation between output samples.

A good answer to “What do the off-diagonal entries mean?” is:

“They measure covariance between different output sample positions. Filtering generally makes those samples statistically dependent at second order.”

This relationship also appears in:

- Linear estimation
- Wiener filtering
- Kalman filtering
- Sensor fusion
- Uncertainty propagation
- Whitening and PCA
- Gaussian-process models

The scalar autocorrelation equations are the infinite-length, Toeplitz counterpart of the finite matrix equation.

Slide 13: White Noise and Filtering

White noise is defined through its second-order properties.

For zero-mean white noise,

$$r_{XX}[m] = \sigma_X^2 \delta[m].$$

Therefore,

$$r_{XX}[0] = \sigma_X^2$$

and

$$r_{XX}[m] = 0, \quad m \neq 0.$$

The samples are uncorrelated at nonzero lags.

This does not necessarily mean they are statistically independent. Independence implies zero correlation under suitable moment conditions, but zero correlation does not generally imply independence.

For jointly Gaussian variables, uncorrelatedness does imply independence. This is one reason Gaussian white noise is mathematically convenient.

A good answer to “Does white imply Gaussian?” is:

“No. White describes the autocorrelation or power spectrum. Gaussian describes the amplitude distribution.”

A simple non-Gaussian white-noise example is a random sign process:

$$P(X[n] = 1) = P(X[n] = -1) = \frac{1}{2},$$

with independent samples.

It has

$$E\{X[n]\} = 0$$

and

$$r_{XX}[m] = \delta[m],$$

but its amplitude distribution is discrete rather than Gaussian.

Slide 14: White Noise Autocorrelation

The autocorrelation of zero-mean white noise contains only one nonzero sample:

$$r_{XX}[m] = \sigma_X^2 \delta[m].$$

This means white noise has no linear second-order predictability across time.

Knowing $X[n - 1]$ does not improve the best linear prediction of $X[n]$.

A useful comparison is an autoregressive process:

$$X[n] = aX[n - 1] + W[n], \quad |a| < 1,$$

where $W[n]$ is white noise.

The resulting process has an autocorrelation of the form

$$r_{XX}[m] = \sigma_X^2 a^{|m|}$$

for real positive a .

If a is close to one, the autocorrelation decays slowly, so the process retains memory for many samples.

If a is close to zero, the autocorrelation decays quickly.

A good interpretation is:

“The width of the autocorrelation indicates how long the process retains second-order memory.”

A common misconception is that a white-noise waveform must look visually uniform. Whiteness concerns the spectrum and correlation, not the appearance of one short realization.

Slide 15: Filtering White Noise Creates Colored Noise

For white input noise,

$$r_{XX}[m] = \sigma_X^2 \delta[m].$$

Substitute this into the output autocorrelation relationship:

$$r_{YY}[m] = \sigma_X^2 r_{hh}[m].$$

The output autocorrelation has the shape of the filter's deterministic autocorrelation.

At zero lag,

$$\sigma_Y^2 = r_{YY}[0] = \sigma_X^2 \sum_k |h[k]|^2.$$

A moving-average filter makes adjacent output samples similar because neighboring outputs contain many of the same input samples.

For example, with

$$h[n] = \frac{1}{2} \delta[n] + \frac{1}{2} \delta[n - 1],$$

the output is

$$Y[n] = \frac{1}{2} X[n] + \frac{1}{2} X[n - 1].$$

The output variance is

$$\sigma_Y^2 = \sigma_X^2 \left[\left(\frac{1}{2} \right)^2 + \left(\frac{1}{2} \right)^2 \right] = \frac{\sigma_X^2}{2}.$$

The lag-one autocorrelation is

$$r_{YY}[1] = \frac{\sigma_X^2}{4}.$$

The filter reduces total variance but introduces memory.

A good answer to “How can a filter reduce variance while creating correlation?” is:

“The filter averages independent input samples, which reduces their combined variance. Neighboring output averages overlap, so the outputs become correlated.”

Slide 16: Power Spectral Density

The power spectral density is the DTFT of the autocorrelation:

$$S_{XX}(e^{j\omega}) = \sum_{m=-\infty}^{\infty} r_{XX}[m]e^{-j\omega m}.$$

The output power spectral density is

$$S_{YY}(e^{j\omega}) = S_{XX}(e^{j\omega})|H(e^{j\omega})|^2.$$

This is one of the most important relationships in random-signal processing.

For white noise,

$$S_{XX}(e^{j\omega}) = \sigma_X^2.$$

Therefore,

$$S_{YY}(e^{j\omega}) = \sigma_X^2 |H(e^{j\omega})|^2.$$

The output noise spectrum follows the squared magnitude response of the filter.

For the two-sample moving average,

$$H(e^{j\omega}) = \frac{1}{2}(1 + e^{-j\omega}).$$

Using

$$1 + e^{-j\omega} = 2e^{-j\omega/2}\cos(\omega/2),$$

we obtain

$$H(e^{j\omega}) = e^{-j\omega/2}\cos(\omega/2).$$

Thus,

$$|H(e^{j\omega})|^2 = \cos^2(\omega/2).$$

For white noise with variance σ_X^2 ,

$$S_{YY}(e^{j\omega}) = \sigma_X^2 \cos^2(\omega/2).$$

At DC,

$$S_{YY}(e^{j0}) = \sigma_X^2.$$

At Nyquist,

$$S_{YY}(e^{j\pi}) = 0.$$

The noise is now lowpass colored.

A good answer is:

“The filter has not made the noise deterministic. It has redistributed the noise power across frequency.”

Slide 17: Noise Augmentation as a Modeling Decision

Noise augmentation should represent nuisance variations expected during deployment.

Adding arbitrary random samples may produce unrealistic training examples. The augmentation should reflect plausible:

- Amplitude distributions
- Spectral shapes
- Correlation times
- SNR ranges
- Transient behavior
- Sensor or channel mechanisms

White Gaussian noise is a useful baseline because it is simple, but it may not resemble real interference.

Examples of colored or structured noise include:

- Low-frequency microphone rumble
- Narrowband electrical hum
- Reverberant background sound
- Slowly varying sensor drift
- Machinery vibration
- Communication-channel interference

A good answer to “What makes an augmentation useful?” is:

“It changes nuisance factors that may vary during deployment while preserving the correct class label.”

A stronger answer adds:

“The augmented training distribution should cover realistic operating conditions without creating physically impossible examples or removing class-defining information.”

Slide 18: Synthetic Noise Augmentation in PyTorch

Basic white-noise augmentation can be written as:

```
noise = torch.randn_like(x)
x_augmented = x + alpha * noise
```

The scale α should usually follow a desired SNR or severity distribution.

Let the clean signal power estimate be

$$P_X = \frac{1}{N} \sum_{n=0}^{N-1} |x[n]|^2.$$

For a target SNR in dB,

$$\text{SNR}_{\text{dB}} = 10 \log_{10} \left(\frac{P_X}{P_N} \right).$$

The desired noise power is

$$P_N = \frac{P_X}{10^{\text{SNR}_{\text{dB}}/10}}.$$

If raw generated noise $n_0[n]$ has measured power

$$P_{N_0} = \frac{1}{N} \sum_n |n_0[n]|^2,$$

scale it using

$$\alpha = \sqrt{\frac{P_N}{P_{N_0}}}.$$

Then

$$n[n] = \alpha n_0[n].$$

A PyTorch implementation is:

```
def add_noise_at_snr(x, noise, snr_db, eps=1e-12):
    signal_power = x.square().mean(dim=-1, keepdim=True)
    noise_power = noise.square().mean(dim=-1, keepdim=True)

    target_ratio = 10.0 ** (snr_db / 10.0)
    desired_noise_power = signal_power / target_ratio

    scale = torch.sqrt(
        desired_noise_power / noise_power.clamp_min(eps))
```

)

```
return x + scale * noise
```

A good answer to “Why scale by power after generating the noise?” is:

“A random finite noise vector will not have exactly the requested RMS. Measuring and rescaling it makes the achieved SNR much closer to the target.”

Only the training data should receive random augmentation. Validation and test data should remain fixed so that model comparisons use the same examples.

Slide 19: White and Colored Augmentation

To create colored noise, begin with white noise and filter it:

```
white = torch.randn_like(x)
colored = apply_filter(white, coefficients)
```

After filtering, normalize the colored noise to the desired RMS before adding it to the clean signal.

This step matters because filtering changes noise power:

$$\sigma_{\text{colored}}^2 = \sigma_{\text{white}}^2 \sum_k |h[k]|^2$$

for a white input and an FIR filter.

If white and colored noise are compared without RMS normalization, any performance difference may result from unequal total noise power rather than correlation or spectral shape.

A controlled experiment should:

1. Use the same clean signals.
2. Give white and colored noise the same RMS or SNR.
3. Compare autocorrelation and PSD.
4. Train with each augmentation.
5. Test under both matched and mismatched noise.

A useful experiment matrix is:

Training augmentation	Test noise	Main question
None	White	Baseline robustness
White	White	Benefit under matched white noise

Training augmentation	Test noise	Main question
White	Colored	Generalization to mismatched noise
Colored	Colored	Benefit under matched colored noise
Colored	White	Whether coloring reduced broader robustness

Good discussion answers:

What should happen to the ACF?

White noise should have a narrow sample autocorrelation concentrated near lag zero. Colored noise should exhibit nonzero correlation over lags determined by the filter.

What should happen to the PSD?

White noise should be approximately flat. Colored noise should follow the filter's squared magnitude response.

Why listen to or inspect the signals if the mathematics is known?

Finite realizations can expose clipping, filter transients, excessive severity, and other implementation problems that theoretical statistics do not reveal.

What does matched augmentation mean?

The augmentation statistics resemble the nuisance statistics in the test or deployment environment.

Slide 20: Exit Question

White noise with variance

$$\sigma_X^2 = 4$$

drives the filter

$$h[n] = \frac{1}{2}\delta[n] + \frac{1}{2}\delta[n-1].$$

For white input noise,

$$\sigma_Y^2 = \sigma_X^2 \sum_k |h[k]|^2.$$

Therefore,

$$\sigma_Y^2 = 4 \left[\left(\frac{1}{2}\right)^2 + \left(\frac{1}{2}\right)^2 \right].$$

Thus,

$$\sigma_Y^2 = 4 \left(\frac{1}{4} + \frac{1}{4} \right) = 2.$$

A good answer is:

“The output variance is 2 because the filter energy is 1/2.”

A stronger answer adds the output correlation:

$$r_{YY}[0] = 2$$

and

$$r_{YY}[1] = r_{YY}[-1] = 1.$$

Therefore,

$$r_{YY}[m] = 2\delta[m] + \delta[m-1] + \delta[m+1].$$

The corresponding output PSD is

$$S_{YY}(e^{j\omega}) = 4\cos^2(\omega/2).$$

A common incorrect answer is

$$\sigma_Y^2 = 4 \left(\frac{1}{2} + \frac{1}{2} \right)^2 = 4.$$

That calculation treats the two input samples as perfectly correlated. White-noise samples at different times are uncorrelated, so variances add using squared coefficients:

$$\text{var}(aX + bY) = |a|^2 \text{var}(X) + |b|^2 \text{var}(Y)$$

when X and Y are uncorrelated.

Slide 21: References and Instructor Resources

The central mathematical results from this lecture are:

Output mean

$$\mu_Y = \mu_X H(e^{j0}) = \mu_X \sum_k h[k].$$

General output variance

$$\sigma_Y^2 = \sum_k \sum_\ell h[k] h^*[\ell] c_{XX}[k - \ell].$$

White-noise output variance

$$\sigma_Y^2 = \sigma_X^2 \sum_k |h[k]|^2.$$

Input-output correlation for the lecture convention

$$r_{XY}[m] = (r_{XX} * h^*)[m].$$

For real $h[n]$,

$$r_{XY}[m] = (r_{XX} * h)[m].$$

Output autocorrelation for a real filter

$$r_{YY}[m] = r_{XX}[m] * h[m] * h[-m].$$

Output power spectral density

$$S_{YY}(e^{j\omega}) = S_{XX}(e^{j\omega}) |H(e^{j\omega})|^2.$$

The larger engineering lesson is:

“Filtering changes both the total noise power and the statistical structure of the noise. An ML model trained with synthetic noise should see the kinds of correlation and spectral coloring expected during deployment.”

A useful graduate-level extension is to derive the output PSD directly from the double-sum expression for $r_{YY}[m]$, then show that taking the DTFT converts convolution into multiplication and produces the factor $|H(e^{j\omega})|^2$.

Week 8, Lectures 15 and 16, Exit Tickets

What are good exit ticket questions with answers for these lectures?

Gemini Pro:

Lecture 15: Characterizing Random Signals

- **Question:** A process has constant mean and $r_{xx}[n_1, n_2] = \exp(-0.2|n_2 - n_1|)$. What property makes it WSS?
- **Answer:** Its mean is constant and its autocorrelation depends only on the lag $n_2 - n_1$, not on absolute time.
- **Question:** A process has constant mean but variance $\sigma^2[n] = 1 + 0.01n$. Is it WSS?

- **Answer:** No. WSS requires the second-order statistics to be time-shift invariant. A variance that changes with n violates that requirement even though the mean is constant.
- **Question:** Why must an autocorrelation matrix be positive semidefinite?
- **Answer:** For any coefficients a , the quadratic form $a^H R_x a$ equals the expected squared magnitude $E\{|\sum a_k^* X[k]|^2\}$. A squared magnitude cannot have a negative expectation.

Lecture 16: LTI Systems with Random Inputs

- **Question:** White noise with variance 4 drives the filter $h[n] = \{1/2, 1/2\}$. What is the output variance?
- **Answer:** $\sigma_y^2 = 4[(1/2)^2 + (1/2)^2] = 2$. The lowpass filter reduces the total noise variance while introducing correlation.
- **Question:** Input $X[n]$ has mean 3. The filter is $h[n] = \{1, -1\}$. What is the output mean μ_y ?
- **Answer:** $\mu_y = 3(1 - 1) = 0$. The first-difference filter has zero DC gain, so it completely removes the constant mean component.
- **Question:** If $h[n] = \delta[n] + 0.5\delta[n - 1]$, does a white-noise input remain white at the output?
- **Answer:** No. Adjacent output samples share one of the same input samples, which causes them to become correlated. The filter successfully "colors" the white noise.

Week 8, Lectures 15 and 16, Exit Tickets

What are good exit ticket questions with answers for these lectures?

ChatGPT:

Week 8 Exit Tickets

A practical exit ticket is two or three questions that students can complete in about five minutes. Each ticket should include:

- One definition or interpretation question
- One short calculation
- One question that exposes a common misconception

Lecture 15: Characterizing Random Signals

Recommended exit ticket

Question 1: Testing for WSS

A random process has

$$E\{X[n]\} = 2$$

and

$$r_{XX}[n_1, n_2] = 4 + 3(0.6)^{|n_2 - n_1|}.$$

Is the process WSS? Explain.

Answer

Yes. The mean is constant, and the autocorrelation depends only on the lag

$$m = n_2 - n_1,$$

rather than on the absolute values of n_1 and n_2 .

Therefore,

$$r_{XX}[m] = 4 + 3(0.6)^{|m|}.$$

The process satisfies the first-order and second-order stationarity requirements.

Strong answer

A strong answer explicitly mentions both WSS conditions. Simply writing “yes, because the autocorrelation is stationary” is too vague.

Question 2: Variance from Autocorrelation

For the process in Question 1, find its variance.

Answer

At zero lag,

$$r_{XX}[0] = 4 + 3 = 7.$$

Since

$$r_{XX}[0] = \sigma_X^2 + |\mu_X|^2,$$

the variance is

$$\sigma_X^2 = 7 - 2^2 = 3.$$

Common incorrect answer

A student may report

$$\sigma_X^2 = 7.$$

That treats the zero-lag autocorrelation as the variance. They are equal only for a zero-mean process.

Question 3: Stationarity and Ergodicity

A sensor supplies one long realization of a WSS process. Does WSS alone guarantee that its time average equals the ensemble mean?

Answer

No. WSS makes the ensemble mean constant and the autocorrelation dependent only on lag. Ergodicity is the additional condition that justifies estimating ensemble statistics from time averages of one realization.

Strong answer

“WSS gives time-invariant first- and second-order ensemble statistics. Mean ergodicity is needed for the time average of one sufficiently long realization to converge to the ensemble mean.”

Alternative Lecture 15 questions

Question 4: Random Process Versus Realization

A microphone records 10 seconds of thermal noise. Is the stored waveform a random process or a realization?

Answer

The stored waveform is one realization, or sample path. The random process is the probabilistic model describing all possible recordings generated under the same conditions.

Question 5: Time-Varying Variance

A process has constant mean zero and

$$E\{|X[n]|^2\} = 1 + 0.02n.$$

Is it WSS?

Answer

No. Since the mean is zero,

$$r_{XX}[0] = E\{|X[n]|^2\} = 1 + 0.02n.$$

The zero-lag autocorrelation depends on absolute time, so the process is not WSS.

Question 6: White Noise Autocorrelation

A zero-mean white-noise process has variance $\sigma^2 = 5$. Write its autocorrelation.

Answer

$$r_{XX}[m] = 5\delta[m].$$

It equals 5 at zero lag and zero at every nonzero lag.

Question 7: Interpreting Autocorrelation

Process A has an autocorrelation that falls rapidly to zero. Process B has an autocorrelation that remains large over many lags. Which process has longer statistical memory?

Answer

Process B has longer statistical memory because samples remain correlated over larger separations.

A strong answer avoids saying that future samples are completely determined by past samples. Correlation indicates second-order statistical dependence, not exact predictability.

Question 8: Cross-Correlation and Delay

Use the convention

$$r_{XY}[m] = E\{X[n]Y^*[n+m]\}.$$

The cross-correlation has its largest peak at $m = 12$. What does this suggest?

Answer

It suggests that $Y[n]$ is delayed relative to $X[n]$ by approximately 12 samples.

If

$$Y[n] = X[n - d],$$

then

$$r_{XY}[m] = r_{XX}[m - d],$$

which peaks at $m = d$.

A strong answer states that this interpretation depends on the stated lag convention.

Question 9: Correlation Matrix Structure

A real WSS process has

$$r_{XX}[0] = 2, \quad r_{XX}[1] = 0.5, \quad r_{XX}[2] = 0.1.$$

Write the correlation matrix for the vector

$$\mathbf{x} = [X[0] \quad X[1] \quad X[2]]^T.$$

Answer

$$\mathbf{R}_X = \begin{bmatrix} 2 & 0.5 & 0.1 \\ 0.5 & 2 & 0.5 \\ 0.1 & 0.5 & 2 \end{bmatrix}.$$

It is symmetric and Toeplitz.

Question 10: Positive Semidefinite Property

Why must an autocorrelation matrix be positive semidefinite?

Answer

For every coefficient vector $\mathbf{a}$,

$$\mathbf{a}^H \mathbf{R}_X \mathbf{a} = E\{|\mathbf{a}^H \mathbf{x}|^2\}.$$

A squared magnitude cannot be negative, so

$$\mathbf{a}^H \mathbf{R}_X \mathbf{a} \geq 0.$$

Lecture 16: LTI Systems with Random Inputs

Recommended exit ticket

Question 1: Output Mean

A WSS input has mean

$$\mu_X = 4.$$

It passes through the FIR filter

$$h[n] = 0.25\delta[n] + 0.5\delta[n - 1] + 0.25\delta[n - 2].$$

Find the output mean.

Answer

The DC gain is

$$H(e^{j0}) = \sum_k h[k] = 0.25 + 0.5 + 0.25 = 1.$$

Therefore,

$$\mu_Y = \mu_X H(e^{j0}) = 4.$$

Strong answer

“The output mean remains 4 because the filter has unit DC gain.”

Question 2: Output Variance for White Noise

Zero-mean white noise with variance

$$\sigma_X^2 = 4$$

passes through

$$h[n] = \frac{1}{2}\delta[n] + \frac{1}{2}\delta[n - 1].$$

Find the output variance.

Answer

For white noise,

$$\sigma_Y^2 = \sigma_X^2 \sum_k |h[k]|^2.$$

Thus,

$$\sigma_Y^2 = 4 \left[\left(\frac{1}{2} \right)^2 + \left(\frac{1}{2} \right)^2 \right] = 2.$$

Common incorrect answer

$$4 \left(\frac{1}{2} + \frac{1}{2} \right)^2 = 4.$$

This calculation treats the two input samples as perfectly correlated. White-noise samples at different indices are uncorrelated, so their variances add using squared coefficients.

Question 3: Does the Output Remain White?

For the system in Question 2, does the output remain white? Explain.

Answer

No. Adjacent outputs share one input sample:

$$Y[n] = \frac{1}{2}X[n] + \frac{1}{2}X[n-1]$$

and

$$Y[n+1] = \frac{1}{2}X[n+1] + \frac{1}{2}X[n].$$

Therefore,

$$r_{YY}[1] = \frac{\sigma_X^2}{4} = 1.$$

The output has nonzero correlation at lag one, so it is colored noise.

Strong answer

“The filter reduces the total variance but introduces correlation because adjacent output averages overlap.”

Alternative Lecture 16 questions

Question 4: First-Difference Filter

A WSS process with mean $\mu_X = 6$ passes through

$$h[n] = \delta[n] - \delta[n-1].$$

Find the output mean.

Answer

$$\mu_Y = \mu_X \sum_k h[k] = 6(1 - 1) = 0.$$

The first-difference filter removes the constant mean because its DC gain is zero.

Question 5: Variance of a Difference of White Noise

Zero-mean white noise with variance σ_X^2 passes through

$$h[n] = \delta[n] - \delta[n - 1].$$

Find the output variance.

Answer

$$\sigma_Y^2 = \sigma_X^2(|1|^2 + |-1|^2) = 2\sigma_X^2.$$

The difference filter removes DC but increases the variance of white noise by a factor of two.

This is a useful reminder that removing the mean does not necessarily reduce random fluctuation power.

Question 6: Input-Output Correlation

Using

$$r_{XY}[m] = E\{X[n]Y^*[n + m]\},$$

and

$$Y[n] = h[n] * X[n],$$

write $r_{XY}[m]$ in terms of $r_{XX}[m]$ and $h[n]$.

Answer

$$r_{XY}[m] = (r_{XX} * h^*)[m].$$

For a real-valued filter,

$$r_{XY}[m] = (r_{XX} * h)[m].$$

A strong answer includes the conjugation and states the correlation convention.

Question 7: Identity-System Check

If

$$h[n] = \delta[n],$$

what should the input-output correlation equal?

Answer

Since $Y[n] = X[n]$,

$$r_{XY}[m] = r_{XX}[m].$$

This provides a useful check on any derived input-output correlation formula.

Question 8: Output PSD

White noise with variance σ_X^2 passes through an LTI system with frequency response $H(e^{j\omega})$. What is the output PSD?

Answer

White-noise PSD is

$$S_{XX}(e^{j\omega}) = \sigma_X^2.$$

Therefore,

$$S_{YY}(e^{j\omega}) = \sigma_X^2 |H(e^{j\omega})|^2.$$

The filter shapes the initially flat noise spectrum according to its squared magnitude response.

Question 9: Covariance Matrix Propagation

A random vector satisfies

$$\mathbf{y} = \mathbf{H}\mathbf{x}.$$

If $\mathbf{x}$ has covariance $\mathbf{C}_X$, what is $\mathbf{C}_Y$?

Answer

$$\mathbf{C}_Y = \mathbf{H}\mathbf{C}_X\mathbf{H}^H.$$

If the input is white,

$$\mathbf{C}_X = \sigma_X^2 \mathbf{I},$$

then

$$\mathbf{C}_Y = \sigma_X^2 \mathbf{H} \mathbf{H}^H.$$

Question 10: White Versus Gaussian

Does white noise have to be Gaussian?

Answer

No. “White” describes the second-order correlation or PSD. “Gaussian” describes the amplitude distribution.

Independent random samples taking values +1 and −1 with equal probability form a non-Gaussian white-noise process.

Question 11: Equal-RMS Noise Augmentation

Why should white and colored augmentation noise be given the same RMS when comparing their effects on a classifier?

Answer

Equal RMS controls total noise power. Performance differences can then be attributed more clearly to temporal correlation or spectral shape rather than one noise type simply being stronger.

Question 12: Training-Only Augmentation

Why should randomly generated noise augmentation be applied to training data but not to validation data?

Answer

Training augmentation encourages robustness to nuisance variation. Validation data should remain fixed so that every model receives the same evaluation conditions.

A strong answer also notes that a separate fixed noisy test set can be used to evaluate robustness.

Suggested Short Versions

Lecture 15 three-minute ticket

1. State the two WSS conditions.
2. Does WSS guarantee that a time average equals an ensemble average?
3. What does a cross-correlation peak at a nonzero lag indicate?

Lecture 16 three-minute ticket

1. A WSS input has mean 3 and $\sum_k h[k] = 0$. Find μ_Y .
2. White noise with variance 4 passes through $h[n] = 1/2, 1/2$. Find σ_Y^2 .
3. Explain why the output of the filter in Question 2 is colored.

Simple Grading Guide

Use two points per question:

- **2 points:** Correct conclusion with the relevant mathematical or physical justification.
- **1 point:** Correct conclusion but incomplete justification, or a minor calculation error with a correct method.
- **0 points:** Incorrect conclusion or no evidence of the governing concept.

The most revealing answers usually require students to write one equation and one sentence interpreting it.
